

AI Engine e Indicizzazione Semantica

1. Introduzione e Contesto

L'AI Engine rappresenta il componente della Piattaforma Zeta dedicato all'indicizzazione semantica dei documenti e all'interazione conversazionale con gli utenti. Si tratta di un insieme di microservizi progettati per trasformare documenti non strutturati, come email PEC, PDF e allegati di vario formato, in conoscenza interrogabile attraverso un'interfaccia di tipo chat.

Poiché la Piattaforma Zeta mira a centralizzare l'ecosistema dei servizi Aruba in un unico portale, integrando PEC, firma digitale e conservazione, l'indicizzazione semantica, come passo successivo alla tradizionale ricerca testuale, trasforma questo aggregatore in un assistente intelligente che riduce drasticamente il tempo necessario per reperire informazioni, estrarre automaticamente dati chiave dai documenti e abilitare nuovi servizi a valore aggiunto basati sull'analisi documentale.

2. Requisiti e Vincoli

2.1 Vincoli dimensionali

Tuttavia, bisogna relazionarsi con dei requisiti dimensionali significativi che influenzano profondamente le scelte architetturali. In primo luogo, la piattaforma deve essere in grado di integrare circa due milioni di caselle PEC e gestire un volume di oltre cinque milioni di messaggi al giorno, a cui si aggiungono circa cinquanta gigabyte giornalieri di documenti inviati in conservazione, oltre che ai documenti che gli utenti possono caricare direttamente dal portale.

Questo si traduce in una necessità del sistema di sostenere un enorme volume di dati.

2.2 Vincolo di compliance normativa: realizzazione on-premise

Non essendo consentito l'utilizzo di cloud provider esterni né di servizi AI di terze parti, bisogna affidarsi a modelli e tecnologie open-source.

Per quanto riguarda i modelli: LLaMA, Mistral o Qwen, invece per i Database vettoriali: Milvus, Qdrant o l'estensione pgvector di PostgreSQL.

2.3 Requisiti funzionali

L'indicizzazione semantica è prevista come funzionalità opzionale per cliente. Quando attivata, deve indicizzare tutti i documenti dell'utente, sia quelli storicizzati permanentemente sia quelli utilizzati solo operativamente; suppongo, quindi, l'impiego di un'interfaccia di chat che permetta all'utente di interrogare i propri documenti in linguaggio naturale, rispettando rigorosamente l'isolamento dei dati tra clienti diversi.

3. Approcci all'Indicizzazione Semantica

3.1 Ricerca full-text tradizionale

La ricerca full-text rappresenta l'approccio più consolidato all'indicizzazione documentale. Tecnologie come OpenSearch o Elasticsearch indicizzano i documenti come insiemi di termini e utilizzano algoritmi statistici, in particolare BM25, per calcolare la rilevanza dei risultati in base alla frequenza dei termini cercati.

Questo approccio presenta vantaggi significativi per la Piattaforma Zeta. Si integra nativamente con lo stack ELK già previsto per il logging, gestisce volumi nell'ordine dei miliardi di documenti con performance prevedibili, ed eccelle nelle ricerche tecniche basate su identificativi precisi, mittenti, oggetti delle PEC o intervalli temporali.

Tuttavia, poiché, la ricerca full-text presenta limitazioni intrinseche nella comprensione semantica, nella soluzione proposta, quest'ultima viene utilizzata in combinazione con la ricerca semantica, non in alternativa.

3.2 Soluzioni di machine learning tradizionale

Accanto alla ricerca testuale, è possibile impiegare modelli di machine learning classici per arricchire i documenti con metadati strutturati. Si tratta di modelli addestrati per task specifici come la classificazione per tipologia documentale, l'estrazione di entità nominate o il riconoscimento di pattern ricorrenti come codici fiscali, partite IVA o importi.

Questi modelli permettono di etichettare automaticamente ogni documento con informazioni quali il tipo (fattura, contratto, atto giudiziario), il soggetto emittente, gli importi rilevanti e le date significative. Tali metadati possono poi essere indicizzati nel motore full-text e utilizzati come filtri nelle ricerche.

3.3 Large Language Model on-premise

Data la natura on-premise della soluzione, la scelta ricade necessariamente su modelli open-source. Tra le opzioni più adatte al contesto si annoverano Mistral 7B Instruct, Llama 3.1 8B e Qwen 2.5, tutti caratterizzati da un buon equilibrio tra qualità delle risposte e requisiti computazionali. Modelli di dimensioni maggiori, come Llama 70B, offrono prestazioni superiori ma richiedono risorse GPU significativamente più elevate.

Data la natura on-premise della soluzione, il modello non può e non deve essere addestrato sui dati dei clienti.

Il modello rimane generico e i documenti vengono forniti come contesto al momento della query attraverso il pattern RAG.

3.4 Il pattern RAG

Il Retrieval Augmented Generation rappresenta il pattern architetturale principale scelto per la Piattaforma Zeta. RAG combina la capacità di recupero informazioni dei database con la capacità generativa dei modelli linguistici. Invece di insegnare al modello tutti i documenti, questi vengono recuperati e forniti come contesto al momento di ogni domanda.

Il pattern si articola in tre fasi distinte. La prima fase, detta di indicizzazione, avviene in modo asincrono quando i documenti vengono caricati nel sistema. Ogni documento viene suddiviso in segmenti di dimensione gestibile, detti chunk, e ciascun segmento viene trasformato in una rappresentazione vettoriale densa, detta embedding, attraverso un modello specializzato. Questi vettori vengono poi memorizzati in un database ottimizzato per la ricerca di similarità.

La seconda fase, detta di retrieval, avviene al momento della query. La domanda dell'utente viene anch'essa trasformata in un vettore e si cercano nel database i chunk i cui vettori sono più simili. Questo permette di identificare i passaggi documentali semanticamente più rilevanti per la domanda posta, indipendentemente dalla presenza di parole chiave esatte.

La terza fase, detta di generation, combina la domanda originale con i chunk recuperati in un prompt che viene sottoposto al modello linguistico. Il modello genera quindi una risposta basandosi esclusivamente sui documenti forniti nel contesto, citando le fonti da cui ha estratto le informazioni.

Questo approccio offre vantaggi determinanti come la scalabilità, che è garantita dal fatto che i documenti rimangono nel database e il modello lavora solo sui contenuti rilevanti per ciascuna query, il rispetto della sovranità sui dati, che è assicurato dal mantenimento di tutti i dati e modelli on-premise, infine, la tracciabilità è garantita dalla possibilità di indicare sempre da quali documenti provengono le informazioni fornite.

Per migliorare ulteriormente la qualità dei risultati, è consigliabile adottare un approccio ibrido che combini la ricerca semantica con quella full-text. La ricerca puramente vettoriale può infatti mancare termini molto specifici come codici o nomi propri, che vengono invece catturati efficacemente dalla ricerca keyword tradizionale. I risultati delle due modalità vengono poi fusi e riordinati attraverso tecniche di re-ranking.

4. Schema Architettuale On-Premise

4.1 Principi architetturali

L'architettura proposta segue i principi dell'event-driven architecture e dei microservizi. La comunicazione tra componenti avviene prevalentemente in modo asincrono attraverso un message broker, garantendo disaccoppiamento e resilienza. Ogni servizio ha una responsabilità specifica e può essere scalato indipendentemente dagli altri.

4.2 Componenti del sistema

Il Document Ingestion Service costituisce il punto di ingresso per tutti i documenti che devono essere indicizzati. Riceve eventi dai servizi operativi, quali PEC, Conservazione e

portale web, quando vengono generati nuovi documenti. Il servizio si occupa di estrarre il contenuto testuale dai vari formati supportati, normalizzare il testo e arricchirlo con metadati tecnici quali l'identificativo dell'utente, del tenant e del servizio di origine. I documenti normalizzati vengono quindi pubblicati sul message broker per le fasi successive.

Il Semantic Indexer Service consuma gli eventi di documenti pronti per l'indicizzazione e orchestra il processo di trasformazione in conoscenza ricercabile. Esegue la suddivisione del testo in chunk di dimensione appropriata, tipicamente alcune centinaia di token ciascuno con una sovrapposizione per preservare il contesto ai confini. Eventualmente è possibile, invocare i servizi di arricchimento ML per classificazione ed estrazione entità.

L'Embedding Service espone un'interfaccia per la trasformazione di testo in vettori numerici. Il servizio deve gestire sia richieste batch durante l'indicizzazione sia richieste singole a bassa latenza durante le query degli utenti.

Il Vector Store costituisce il repository principale per gli embedding e i relativi metadati.

Il Search and Retrieval Service orchestra le ricerche combinando la modalità semantica con quella full-text. Riceve le query, calcola l'embedding della domanda, esegue la ricerca di similarità sul vector store e la ricerca keyword su OpenSearch, fonde i risultati con algoritmi come Reciprocal Rank Fusion e applica eventualmente un re-ranking per migliorare la precisione. Restituisce i chunk più rilevanti con i relativi metadati e punteggi di rilevanza.

L'AI Chat Service rappresenta il punto di accesso per l'interfaccia conversazionale. Autentica l'utente validando il token JWT, invoca il Search and Retrieval Service per recuperare i documenti pertinenti alla domanda, costruisce un prompt contestualizzato che include le istruzioni di sistema, la domanda dell'utente e gli estratti documentali recuperati, inoltra il prompt al servizio LLM e restituisce la risposta generata insieme ai riferimenti alle fonti utilizzate.

Il LLM Inference Service ospita il modello linguistico e gestisce le richieste di generazione. Espone un'interfaccia di tipo REST che accetta prompt e restituisce il testo generato.

5. Flussi Principali

5.1 Flusso di indicizzazione

Il flusso di indicizzazione si attiva ogni volta che un nuovo documento entra nel sistema, sia esso un messaggio PEC ricevuto, un documento inviato in conservazione o un file caricato manualmente dall'utente attraverso il portale.

Il servizio sorgente genera un evento che contiene l'identificativo del documento, i riferimenti all'utente e al tenant proprietario, il tipo di documento e la sua origine, e il percorso per accedere al contenuto originale nel document store. Questo evento viene pubblicato sul topic Kafka dedicato agli eventi documentali.

Il Document Ingestion Service consuma l'evento, recupera il documento dallo storage e procede all'estrazione del contenuto testuale. Per i PDF viene utilizzata una libreria di parsing che gestisce sia documenti nativi digitali sia documenti scansionati attraverso OCR. Per le email vengono estratti corpo del messaggio e allegati. Il testo estratto viene normalizzato rimuovendo formattazioni superflue e arricchito con i metadati tecnici disponibili.

Il Semantic Indexer Service riceve il documento normalizzato e procede alla suddivisione in chunk. La strategia di chunking privilegia la coerenza semantica, dividendo preferibilmente per paragrafi o sezioni logiche piuttosto che per numero fisso di caratteri. Ogni chunk mantiene una sovrapposizione con i chunk adiacenti per evitare la perdita di contesto ai confini. Vengono inoltre invocati i modelli di classificazione e NER per arricchire ogni chunk con metadati strutturati.

Per ogni chunk viene richiesto il calcolo dell'embedding al servizio dedicato. I vettori risultanti, insieme ai metadati e al testo originale del chunk, vengono quindi persistiti nel vector store. Parallelamente, il contenuto testuale e i metadati vengono indicizzati in OpenSearch per abilitare la ricerca full-text. Il documento originale, se non già presente, viene archiviato nello storage oggetti.

L'intero flusso è asincrono e disaccoppiato, permettendo ai servizi operativi di continuare a funzionare senza attendere il completamento dell'indicizzazione. Un sistema di monitoraggio traccia lo stato di ogni documento attraverso le varie fasi, gestendo automaticamente i retry in caso di errori temporanei e segnalando i documenti problematici che richiedono intervento manuale.

5.2 Flusso di query conversazionale

Il flusso di query si attiva quando un utente autenticato accede alla sezione di chat del portale e pone una domanda sui propri documenti.

La richiesta arriva all'AI Chat Service corredata del token JWT che identifica l'utente. Il servizio valida il token utilizzando le stesse componenti di sicurezza impiegate nel resto della piattaforma, estraendo l'identificativo utente, il tenant di appartenenza e i permessi specifici, ad esempio quali caselle PEC l'utente è autorizzato a consultare.

La domanda dell'utente viene inviata all'Embedding Service per ottenerne la rappresentazione vettoriale. Questo vettore viene poi utilizzato dal Search and Retrieval Service per eseguire una ricerca di similarità nel vector store. La ricerca è sempre filtrata per tenant e per i permessi specifici dell'utente, garantendo che vengano restituiti solo documenti a cui l'utente ha effettivamente accesso.

Parallelamente viene eseguita una ricerca full-text su OpenSearch con gli stessi filtri di sicurezza. I risultati delle due ricerche vengono fusi utilizzando un algoritmo che combina i punteggi di rilevanza semantica e testuale. I chunk risultanti possono essere ulteriormente

riordinati attraverso un modello di re-ranking che valuta la rilevanza di ciascun chunk rispetto alla domanda originale.

L'AI Chat Service costruisce quindi il prompt da sottoporre al modello linguistico. Il prompt include un'istruzione di sistema che definisce il comportamento atteso, indicando al modello di basarsi esclusivamente sui documenti forniti, di citare sempre le fonti e di dichiarare esplicitamente quando un'informazione non è disponibile. Seguono gli estratti dei chunk recuperati, opportunamente formattati con i riferimenti al documento di origine, e infine la domanda dell'utente.

Il prompt viene inviato al LLM Inference Service che genera la risposta. La generazione avviene preferibilmente in modalità streaming, permettendo all'utente di vedere la risposta mentre viene composta. Una volta completata, la risposta viene restituita al frontend insieme ai riferimenti ai documenti da cui sono state estratte le informazioni, permettendo all'utente di accedere direttamente alle fonti per approfondimenti.

L'intera interazione viene tracciata per finalità di audit, registrando la domanda posta, i documenti consultati, la risposta generata e i tempi di elaborazione.

6. Conclusioni

L'architettura proposta costituisce un incipit per implementazioni più complesse e strutturate: una prima evoluzione potrebbe introdurre il supporto multimodale per l'analisi di immagini contenute nei documenti PDF. Successivamente potrebbero essere abilitate azioni automatiche, permettendo all'utente di richiedere via chat operazioni come la firma di un documento o l'invio di una PEC.

L'architettura a microservizi e l'adozione di standard aperti garantiscono che queste evoluzioni possano essere introdotte incrementalmente, senza richiedere una riprogettazione complessiva del sistema.